

Telemetry event storage

Technical Requirements Document · BidOn Foundation

Parquet on DigitalOcean Spaces

Status: Draft

Date: 2026-09-01

This TRD specifies: the events warehouse. One row per product event, retained as Parquet on DigitalOcean Spaces, flushed every 10 minutes.

Why the warehouse exists	PRD v1 — Goals 1–4; Part II “raw events to a columnar warehouse for funnel joins”
What the warehouse must answer	telemetry-requirements.md §2.A (jobs J3, J4, J5)
Why Parquet on object storage	telemetry-events-store.md
Numbers	telemetry-storage-sizing.md

No production instrumentation ships from this draft. The lake can exist empty; producers are a different piece of work.

1. Scope

- A DigitalOcean Spaces bucket (S3 API) for telemetry events.
- **Protobuf** schemas, one message type per event, in Schema Registry.
- A **Kafka Connect** S3 sink (Confluent Community License — not a paid SKU) that reads those protobuf records and writes **Parquet**.
- **One table per event type** (own prefix, own schema). Day partitions inside each table.
- 10-minute flush, compression.
- Lifecycle retention **per table** (400 d reconciliation tables, 90 d diagnostics), plus a daily aggregate table.
- A DuckDB SQL pack an engineer can run against the bucket.
- A freshness alert on the sink.

The sink reads Confluent-wire **protobuf** already on `telemetry-events` (magic byte + schema id + message). Each record is a registered message type. `sampling_rate` is a field on the message; the sink does not re-sample and does not drop by rate.

If the topic is empty, the lake is empty. That is acceptable for this TRD.

2. Why this store

The PRD’s jobs that need a warehouse are catalog SQL:

Job	Query the lake must support
J3	Fill rate, time-to-fill, render rate, per-source load — <code>sum(1 / sampling_rate)</code> , GROUP BY demand
J4	Reconcile Bidon impressions vs mediator / DSP billing

Job Query the lake must support

J5 Per-DSP loss and render

Requirements §2.A: if a system cannot do those joins, it is not the event store. Minutes of lag are fine.

The lake is downstream of Redpanda. If the sink or Spaces is down, ads still serve (G1/G2).

Who runs the SQL. For this slice those jobs are engineer-mediated: someone runs the saved SQL pack and returns numbers.

3. Pipeline

```
Redpanda telemetry-events (protobuf + Schema Registry)
  |
  ▼
Kafka Connect S3 sink — ParquetFormat, flush every 10 min
  |
  ▼
DigitalOcean Spaces
  events/<event_name>/dt=YYYY-MM-DD/*.parquet    ← one table per event type
  aggregates/dt=YYYY-MM-DD/*.parquet             ← daily rollup (its own table)
  |
  ▼
DuckDB (read_parquet per table, join on envelope keys)
```

Protobuf on the topic is the **bus**. Parquet on Spaces is the **store**.

The existing Connect connector (`docker/kafka-connect` , JSON of `ad-events`) stays as that archive. This warehouse is a **new** connector on `telemetry-events` .

4. Requirements

4.1 Bucket

Item	Requirement
Service	DigitalOcean Spaces (S3 API)
Isolation	Dedicated telemetry bucket, or a dedicated prefix on a bucket that is not the Postgres-backup bucket (<code>infra</code> currently creates <code>\${name_prefix}-backups-*</code>). Backup lifecycle must not delete event objects.
ACL	Private. No public list or get.
Credentials	Spaces access key on the Connect worker only. Not in <code>bidon-sdkapi</code> .
Region	Same Spaces region as existing <code>infra</code> (<code>ams3</code> in current Terraform).

4.2 Protobuf schemas

Strict, per event type. The Parquet columns **are** the protobuf fields.

Item	Requirement
Source	<code>.proto</code> files in this repo (alongside existing <code>proto/</code>). Generated Go for producers; the registry is the runtime source for Connect.

Item	Requirement
Shape	One message type per PRD <code>event_name</code> (<code>AuctionCompleted</code> , <code>DspRequestSent</code> , ...). Shared envelope fields live in a reusable message embedded in every event (not a JSON blob).
Wire	Confluent protobuf: magic byte, schema id, protobuf payload.
Registry	Schema Registry (already in <code>docker-compose-prod.yml</code>). Subject strategy TopicRecordNameStrategy so each message type has its own subject on <code>telemetry-events</code> .
Compatibility	BACKWARD per subject. Additive field numbers only; never reuse a number. Matches PRD “additive within a major.”
Versions	The sink lands every registered schema id. It does not reject old majors. DuckDB reads mixed file versions with <code>union_by_name=true</code> . PRD rationale 6: old SDK (and old producer) schemas persist for quarters.

Envelope fields required on every message (embedded): `event_name`, `event_id`, `event_ts`, `schema_version`, `app_id`, `auction_id`, `session_id`, `sampling_rate`, plus the rest of the PRD common envelope as protobuf fields.

Type-specific fields are first-class columns (`dsp_id`, `http_status`, `clearing_price`, ...), not packed into payload.

4.3 Tables, format, and layout

A table has one schema. Strict protobuf per event type therefore means **one table per `event\_name`**, not one `events` table partitioned by type. Hive `event_name=` as a partition of a unified table would imply shared columns; these messages do not share columns beyond the embedded envelope.

Item	Requirement
Table	One per PRD event type. Name = <code>event_name</code> (<code>auction_completed</code> , <code>dsp_request_sent</code> , ...). ~40 tables at full catalog.
Encoding	Parquet (<code>ParquetFormat</code> , snappy or zstd).
Columns	That type's protobuf fields (envelope + type-specific).
Layout	<code>s3://<bucket>/events/<event_name>/dt=YYYY-MM-DD/*.parquet</code>
Partition clock	<code>dt</code> from <code>event_ts</code> (event wall clock), not ingest wall clock. Late events land on their event day.
Homogeneous files	A file belongs to one table. Connect Parquet cannot hold two schemas.
Joins	Envelope keys (<code>app_id</code> , <code>auction_id</code>) (G10). Funnel SQL is JOIN across tables , not <code>GROUP BY event_name</code> on a union.
Schema evolution	<code>union_by_name</code> within a table (old vs new field numbers of the same message). Do not union different event types into one scan.

Connect partitioner. `FieldPartitioner` on `event_name` (table directory) plus a `dt` field materialised from `event_ts` (SMT `TimestampConverter` or equivalent). One connector writes all tables.

Small files. At 10k DAU a 10-minute flush splits across ~40 tables. Accepted at 10k. Compact **inside** a table if LIST hurts. Do not merge types into one prefix.

Not a table: a fat protobuf `oneof` wrapping every event, or one Parquet schema with every field nullable. That undoes strict per-type schemas.

4.4 Writer

Kafka Connect S3 sink, Confluent Community License (no fee; competing-SaaS restriction only). Redpanda is the bus.

Item	Requirement
Cardinality	One task (<code>tasks.max=1</code>).
Connector	New: <code>telemetry-events-parquet</code> . Class <code>io.confluent.connect.s3.S3SinkConnector</code> .
Format	<code>io.confluent.connect.s3.format.parquet.ParquetFormat</code>
Value converter	<code>io.confluent.connect.protobuf.ProtobufConverter</code> + Schema Registry URL. Not <code>JsonConverter</code> (NPE with <code>ParquetFormat</code>).
Input	Topic <code>telemetry-events</code> only.
Spaces	Path-style, <code>store.url</code> for the Spaces endpoint, keys from Connect env.
Empty flush	Do not write an object when the batch is empty.
Idempotence	At-least-once. Duplicate Parquet rows are a query concern (<code>ROW_NUMBER() OVER (PARTITION BY app_id, event_id)</code>).

Reuse the existing Connect **worker** (`docker/kafka-connect`). Do not retarget the JSON `ad-events` connector.

4.5 Flush

Trigger	Value
Time	Every 10 minutes — <code>rotate.schedule.interval.ms=600000</code> (set <code>timezone=UTC</code>)
Size	<code>flush.size</code> high enough that time wins at 10k DAU (~41k events / 10 min). Revisit once protobuf bytes/row are measured.
Empty	Skip

10-minute lag is acceptable for funnel SQL. Do not flush more often to chase freshness — small files are the cost of that.

4.6 Topic (sink input)

The sink cannot start without a topic. Provisioning it is in this TRD because it is a store prerequisite.

Item	Requirement
Name	<code>telemetry-events</code> (env <code>KAFKA_TELEMETRY_EVENTS_TOPIC</code>)
Creation	Explicit. <code>config/kafka.go</code> allows auto-create, which yields one partition.
Compression	Producer <code>zstd</code> (set at topic / producer; do not rely on the <code>franz-go</code> default).
Retention on the bus	Days , not 400 d. The lake is the retain.
Partitions	Set from peak protobuf/day before first write (sizing §7).

4.7 Query

Item	Requirement
Engine	DuckDB. Scan one table at a time: <code>read_parquet('s3://.../events/ad_impression/dt=.../**', union_by_name=true)</code> . Not an always-on warehouse.

Item	Requirement
When	On demand. Start DuckDB when someone has a question.
Users	Engineers.
SQL pack	Funnel is joins: <code>ad_request_started</code> ⋈ <code>ad_filled</code> ⋈ <code>ad_impression</code> ⋈ <code>billing_notice_sent</code> on <code>(app_id, auction_id)</code> , each from its own table. DSP timeout and notice success similarly. Counts corrected by <code>sum(1 / sampling_rate)</code> .

4.8 Retention

Three drivers, three artefacts. Do not keep one window for everything. Detail: sizing \$4.

Class	Window	Where	Contents
Aggregates	7–10 years	table <code>aggregates/</code>	Daily rollup: <code>date</code> × <code>app</code> × <code>DSP</code> × <code>ad_format</code> × <code>country</code> → impressions, billable amount, notice successes. No ids, no per-user grain. Statutory ledger .
Reconciliation	400 days	those tables only	<code>ad_impression</code> , <code>billing_notice_sent</code> , <code>win_notice_sent</code> , <code>auction_completed</code> , <code>ad_filled</code>
Diagnostics	90 days	every other event table	Including loss notices

Lifecycle is per table prefix, not a row filter inside a mixed `events/` folder. Spaces rules on `events/ad_impression/` vs `events/loss_notice_sent/` (and the other names in each class). Hive cannot `DELETE` a row; it can drop a day of a table.

400 d is an analytics/audit buffer, not a tax derivation. Confirm the contractual dispute window (AppLovin / DSP agreements) before treating it as settled. Statutory 6–10 year accounting is satisfied by the aggregate, not by replaying auctions.

Aggregates ship with the sink. ~10 MB/day, ~25 GiB over seven years. They are computed from never-sampled events, so they do not depend on `K`. They cannot be backfilled once raw rows expire. A scheduled DuckDB (or equivalent) job writes one Parquet file per day to `aggregates/`.

Loss notices sit at 90 d on purpose: only events evidencing a **billing** claim need the reconciliation window.

4.9 Failure isolation

Failure	Ads	Lake
Connect stalled	Unaffected	Freshness alert; bus retains days of protobuf
Schema Registry down	Unaffected (produce may fail — producer concern)	Sink cannot decode new ids until SR returns
Spaces down / 5xx	Unaffected	Connector backs off; Kafka consumer lag grows
DuckDB not running	Unaffected	No one can query until it is started
Topic empty (no producers yet)	Unaffected	Lake empty; freshness check must not false-page

The sink is not on the ad path. Do not add a Spaces, Connect, or DuckDB dependency to `bidon-sdkapi`.

5. Volume and cost

Design point: ~10k DAU, 2 sessions × 5 auctions, fill 0.4, **unsampled** ($K = 1.0$). Model and caveats: telemetry-storage-sizing.md.

Sizing still uses **1.2 KiB/event on the bus** and **200 B/row Parquet** as unmeasured placeholders (those were JSON-era). Protobuf + typed columns should compress better than a `payload` JSON blob; treat figures as $\pm 2\times$ until the sink produces real bytes.

	Auctions / day	Events / day	Parquet / day	On Spaces (90/400 d)	Per 10 min flush	Spaces \$/mo
10k DAU	100k	5.9M	1.1 GiB	114 GiB	~41k events	~\$5
100k	1M	59M	11 GiB	1.1 TiB	~410k	~\$23
1M	10M	590M	110 GiB	11 TiB	~4.1M	~\$230

1.1 GiB/day is new Parquet. 114 GiB is retained. Bus retention is days, not 400 d.

First measurement after the sink writes: mean protobuf bytes on the wire, Parquet bytes/row, and `event_name` count per `(app_id, auction_id)`.

6. Sink observability

Signal	Purpose
Consumer lag on <code>telemetry-events</code>	Bus is backing up
Connect task state / failed records	Decoder or PUT errors
Time since last successful Spaces PUT	Writer is stalled
Latest object <code>dt</code> / modified time vs topic high-water	Freshness
Objects written / bytes written per flush	Confirms the 10 min trigger

Alert: latest object lags the topic by **> 20 minutes** (two flush intervals). Do not fire that alert when the topic has had no records in the window.

7. Deliverables

#	Work	Done when
1	Spaces bucket (or dedicated prefix) + private credentials	Terraform or Coolify; not the backups bucket
2	Schema Registry reachable from Connect (prod compose already has it; dev/Coolify must too)	Protobuf converter can fetch ids
3	Protobuf event messages in-repo + subjects registered (BACKWARD, TopicRecordNameStrategy)	At least one event type round-trips
4	<code>telemetry-events</code> topic created explicitly (partitions, day-scale retention, zstd)	Topic exists before the sink starts
5	New Connect connector <code>telemetry-events-parquet</code> : ProtobufConverter, ParquetFormat, 10 min rotate, one table prefix per <code>event_name + dt=</code> , Spaces endpoint	Objects appear under <code>events/<event_name>/dt=...</code> as Parquet

#	Work	Done when
6	Lifecycle rules per table : 90 d diagnostics tables, 400 d reconciliation tables, 7–10 y aggregates	Day prefixes expire; the two classes differ by table
7	Daily aggregate job writing the <code>aggregates</code> table	One Parquet file per day
8	DuckDB SQL pack (per-table scans, funnel as joins)	An engineer can answer J3/J4 against a day of files
9	Freshness alert (> 20 min, quiet-topic excluded)	Page on stall, not on idle

Local/dev: MinIO or a Spaces staging bucket is enough. Do not require production Spaces to prove the writer.

8. Acceptance

This TRD is done when all of the following are true:

1. A protobuf record on `telemetry-events` (registered schema id) becomes a Parquet row under `events/<event_name>/dt=<event day>/` within **10 minutes** under load where time-based rotate fires first.
2. That file's columns match that event's protobuf message. DuckDB `read_parquet` on **that table** (with `union_by_name` for schema versions of the same type) returns the row. Funnel SQL joins tables on `(app_id, auction_id)` and corrects with `sampling_rate`.
3. Objects are Parquet. The writer is the new Connect connector, consuming `telemetry-events` only. Each `events/<event_name>/` prefix is one table: one message type, one schema.
4. `bidon-sdkapi` has no Spaces, Connect, or DuckDB import; killing Connect does not change auction or `/show` behaviour.
5. Diagnostics tables older than 90 d are gone; reconciliation tables survive to 400 d; `aggregates/` is excluded from those expiries.
6. After one scheduled run, the `aggregates` table contains a rollup file for that day.
7. Freshness pages when the writer is stalled with records on the topic, and does not page when the topic is idle.

9. Open items

Nothing below is architecture-blocked. Every item needs a **name**.

#	Item	Blocks	Owner
1	Who provisions Spaces, Schema Registry (dev/Coolify), and the Connect connector	This TRD entirely	unnamed
2	Contractual dispute / audit window (MAX terms, DSP agreements) — is 400 d right?	Reconciliation lifecycle	Commercial / counsel
3	Measured protobuf bytes/event and Parquet bytes/row	Turns \$5 from a model into a bill	Backend, once the sink writes

10. PRD traceability

PRD requirement	This TRD	Status
Raw events to a columnar warehouse for funnel joins	Entire document	Protobuf → Connect → Parquet on Spaces, DuckDB on demand

PRD requirement	This TRD	Status
Funnel completeness, fill / render / notice rates (catalog SQL)	§4.7 SQL pack	Engineer-mediated in this slice
Goal 3 reconciliation (impression-level rows)	§4.8 400 d class + aggregates	Window unconfirmed commercially
Retention per event class before launch	§4.8	Lifecycle + aggregate job
Schema versioned; additive within a major	§4.2 BACKWARD, additive field numbers	Sink lands every registered id
Telemetry must not delay the ad path	§4.9	Sink is off the ad path
Sampling rate recorded so counts correct at query time	§4.7 <code>sum(1 / sampling_rate)</code>	Field on the protobuf message